

Solicitação de Projeto: Desenvolvimento de um Compilador Completo

1. Objetivo

O objetivo deste projeto é consolidar os conhecimentos teóricos sobre a teoria de linguagens formais e autômatos através da construção prática de um compilador funcional.

2. Etapas de Tradução (Entregáveis)

O projeto deve ser dividido nas seguintes fases obrigatórias:

A. Análise Léxica (Scanner)

O aluno deve implementar um analisador que transforme o fluxo de caracteres de entrada em uma sequência de tokens.

- Requisitos: Reconhecimento de palavras reservadas, identificadores, literais (números, strings) e operadores. Deve ignorar espaços em branco e comentários.
- Teoria aplicada: Expressões Regulares e Autômatos Finitos Determinísticos (AFD).

B. Análise Sintática (Parser)

Nesta etapa, o compilador deve agrupar os tokens em estruturas gramaticais, verificando se a ordem dos elementos respeita as regras da linguagem.

- Requisitos: Construção da Árvore de Sintaxe Abstrata (AST).
- Método: Pode ser utilizado um *Parser Descendente Recursivo* ou geradores como Yacc/Bison/ANTLR (conforme critério do professor).

C. Análise Semântica

A análise semântica deve garantir que o programa faça sentido logicamente, além da estrutura gramatical.

- Requisitos:
 - Criação e gerenciamento da Tabela de Símbolos.
 - Verificação de tipos (Type Checking).
 - Declaração prévia de variáveis e escopo.

D. Geração de Código Intermediário (IR)

O código fonte deve ser traduzido para uma representação independente de máquina.

- Requisitos: Geração de Código de Três Endereços (TAC) ou representações similares. Isso facilita a otimização e a portabilidade.

E. Geração de Código Final e Otimização

A etapa final consiste em traduzir a IR para uma linguagem de baixo nível.

- Requisitos: Tradução para Assembly (x86 ou MIPS) ou Bytecode para uma máquina virtual específica.

- Otimização (Opcional/Bônus): Eliminação de código morto ou simplificação de expressões aritméticas.

3. Especificação da Linguagem-Alvo

Para fins didáticos, a linguagem deve suportar ao menos:

1. Tipos de dados: Inteiros e Booleanos.
2. Estruturas de Controle: if-else e laços while.
3. Operações: Aritméticas (+, -, *, /) e Lógicas (==, !=, <, >).
4. E/S: Comandos simples de leitura e escrita (ex: print e read).

4. Critérios de Avaliação

Critério	Peso	Descrição
Corretude Léxica/Sintática	30%	O compilador aceita programas válidos e rejeita inválidos?
Análise Semântica	20%	O sistema detecta erros de tipo e variáveis não declaradas?
Geração de Código	30%	O código final produzido executa e gera o resultado esperado?
Documentação/Código	20%	Qualidade do código, comentários e relatório explicativo.